Episode 2: Install Docker and Your First Commands

Theory Notes - plain language, with a hands-on lab

The big idea: You type simple commands. Docker's background service (the daemon) does the real work: it downloads images, starts containers, and keeps them running. By the end of this episode you can install Docker, run a web server in one command, read its logs, step inside it, and clean up.

1. Installing Docker

Which Docker you install depends on your computer:

System	What to install	Notes
Windows 10/11	Docker Desktop	Uses the WSL 2 backend (a built-in Linux layer). Restart if asked, open Docker Desktop and wait until it says the engine is running.
macOS	Docker Desktop	Choose the Apple silicon or Intel download that matches your Mac.
Linux (for example Ubuntu)	Docker Engine	Install from Docker's official repository. Then add your user to the `docker` group so you do not need sudo.
No install possible	Play with Docker	A free browser playground at labs.play-with-docker.com (needs a Docker Hub account).

Tip: Steps change over time, so always follow the official guide at docs.docker.com for your system. Companies should also check Docker Desktop's licensing terms before using it for work.

Verify the installation (three checks)

```
$ docker --version        # is the client installed?
$ docker info             # is the daemon running and reachable?
$ docker run hello-world  # does the whole flow work?
```

2. How Docker works: waiter and chef

Docker has two main parts. The **docker CLI** (client) is the command you type, like a **waiter** taking your order. The **Docker daemon** (dockerd) is a background service, like the **chef** who cooks. The CLI sends your request to the daemon, which pulls images from a registry, creates containers and manages them. If the daemon is not running, every command fails with a "cannot connect" error.

3. Images: pull and list

```
$ docker pull nginx      # download the nginx image from Docker Hub
$ docker images          # list images on your computer
$ docker rmi nginx       # delete an image (only if no container uses it)
```

nginx is a popular web server. If you skip `pull`, `docker run` downloads the image automatically when it is missing.

4. docker run: the most important command

```
$ docker run -d -p 8080:80 --name web nginx
```

The shape is: `docker run [OPTIONS] IMAGE [COMMAND]`. Options go **before** the image name.

Part	Meaning	Story analogy
`run`	Create a new container from an image and start it.	Cook a new dish
`-d`	Detached: run in the background and give your terminal back.	Cook in the back kitchen
`-p 8080:80`	Publish a port. Format is HOST:CONTAINER.	Window 8080 leads to hatch 80
`--name web`	Give the container a friendly name (otherwise Docker invents one).	Name tag on the dish
`nginx`	The image to start from.	The recipe kit
`-it`	Interactive terminal. Used with shells such as bash.	Sit at the chef's counter
`--rm`	Automatically delete the container when it exits.	Throw away the plate after eating

5. Ports: why -p matters

A container has its own private network. The nginx server inside listens on **port 80**, but your browser cannot reach it unless you **publish** the port to your computer. `-p 8080:80` means: when something visits port **8080 on your computer (host)**, forward it to port **80 inside the container**. Open `http://localhost:8080` and you will see "Welcome to nginx!".

> **Remember the order: HOST : CONTAINER** (outside : inside). Writing `-p 80:8080` by mistake sends host port 80 to container port 8080, so the page will not load.

6. Looking at containers

```
$ docker ps                 # running containers
$ docker ps -a              # all containers, including stopped
$ docker logs web           # what the app printed
$ docker logs -f web        # follow logs live (Ctrl+C to stop following)
$ docker logs --tail 20 web # last 20 lines
$ docker inspect web        # full details as JSON
```

Reading docker ps: the columns are CONTAINER ID, IMAGE, COMMAND, CREATED, STATUS, PORTS and NAMES. In PORTS you should see something like `0.0.0.0:8080->80/tcp`. A second line for IPv6 may also appear.

Step inside: docker exec

```
$ docker exec -it web bash   # open a shell inside the running container
root@web:/# ls
root@web:/# exit
```

`exec` runs an **extra** command inside a container that is already running. If an image has no bash (for example Alpine images), use `sh` instead. Use exec to look around and debug, not to configure your app, because changes are lost when the container is removed.

7. Stop, start and remove

```
$ docker stop web      # stop it gracefully (it moves to Exited)
$ docker start web     # start the same container again
$ docker restart web   # stop then start
$ docker rm web        # delete a stopped container
$ docker rm -f web     # force: stop and delete in one go
```

Container lifecycle: Created, Running, Exited, Removed. `docker run` does create + start in one step. `docker stop` first asks the app to shut down politely, then forces it after a short wait (10 seconds by default).

rm vs rmi: `rm` removes a **container**. `rmi` removes an **image**.

A container lives only while its main process runs. If that process ends, the container exits. For example `docker run ubuntu` exits at once because nothing keeps it busy.

8. Hands-on lab (10 minutes)

Step	Command	What you should see
1	`docker pull nginx`	Layers downloading, then a Status line.
2	`docker run -d -p 8080:80 --name web nginx`	A long container ID printed.
3	Open `http://localhost:8080`	"Welcome to nginx!" page.
4	`docker ps`	Container web, status Up, ports 8080->80.
5	`docker logs web`	Lines showing your browser's request.
6	`docker exec -it web bash` then `exit`	A prompt inside the container, then back to your terminal.
7	`docker stop web` then `docker rm web`	The name echoed back each time. docker ps -a no longer lists it.

9. Cheat sheet

Goal	Command
Download an image	docker pull IMAGE
List images	docker images
Start a container	docker run -d -p HOST:CONTAINER --name NAME IMAGE
List running / all	docker ps / docker ps -a
Read logs	docker logs -f NAME
Open a shell	docker exec -it NAME bash
Show details	docker inspect NAME
Stop / start	docker stop NAME / docker start NAME
Remove container / image	docker rm NAME / docker rmi IMAGE

10. Mini glossary

Term	Meaning
CLI	The docker command you type.
Daemon	The background service that does the work (dockerd).
Detached mode	Running in the background (-d).
Port mapping	Forwarding a host port to a container port (-p).
Host	Your real computer.
Exited	A stopped container that still exists until removed.
Image tag	The version label after the colon, such as nginx:1.27.

11. Key takeaways

- The CLI (waiter) sends orders to the daemon (chef).
- docker run = create + start. Use -d for background, -p HOST:CONTAINER for ports, --name for a label.
- docker ps, logs, inspect and exec are your four checking tools.
- Stop a container before removing it, or use rm -f. rm is for containers, rmi for images.

12. Self-check quiz

Q1. What is the difference between the Docker CLI and the daemon?
Q2. In `-p 8080:80`, which number is the container's port?
Q3. Which command shows stopped containers too?
Q4. What does `-d` do?
Q5. How do you open a shell inside a running container?

Answers

A1. The CLI is what you type; the daemon is the background service that does the work.
A2. 80 (host port 8080 comes first).
A3. docker ps -a
A4. Runs the container in the background (detached).
A5. docker exec -it NAME bash (or sh).

Homework: Run a second nginx on port 8081 with a different name, then use docker ps to confirm both are running. **Next episode:** Writing your first Dockerfile.